

DISSERTATION DEFENSE REFERENCE GUIDE

Algorithms, Preprocessing, and Examiner Q&A Guide

BITS ZG628T: Dissertation

Student: Harisha P V (ID: 2024AA05069)

Supervisor: Mallinath Patil (Dell Technologies, Bengaluru)

1. Overall Algorithms Used in the Project

The RAG pipeline integrates several state-of-the-art algorithms across retrieval, indexing, reasoning, and routing phases:

1.1 Approximate Nearest Neighbor (ANN) Search: HNSW (Hierarchical Navigable Small World)

- Where it is used: Inside the Qdrant Vector Database.
- What it does: Indexes high-dimensional dense vector embeddings (1024-dimensions) using a multi-layer graph structure. This allows the database to retrieve the top-K candidate tickets in sub-linear time, avoiding exhaustive and slow linear scans.

1.2 Lexical Ranking Algorithm: BM25 (Best Matching 25)

- Where it is used: In the sparse retrieval branch of the hybrid index.
- What it does: A probabilistic ranking framework that calculates the relevance of a candidate ticket based on Term Frequency (TF), Inverse Document Frequency (IDF), and document length normalization. This guarantees exact keyword matching for diagnostic codes and trace identifiers.

1.3 Rank Fusion Algorithm: RRF (Reciprocal Rank Fusion)

- Where it is used: Merging dense semantic and sparse lexical search results.
- What it does: Fuses the two ranked candidate lists based on their rank positions rather than their raw scores (which have incompatible scales). The RRF formula is:

$$\text{RRF_Score}(d) = \sum_{m \in M} \frac{1}{(k + r_m(d))}$$

where $k=60$ is the smoothing constant.

1.4 Text Embedding Representation: Transformer-Based Semantic Encoding

- Where it is used: Encoding logs and tickets into dense vectors via BAAI/bge-large-en-v1.5.
- What it does: Leverages a deep transformer network (BERT-based) trained on massive text corpora to map technical bug summaries into a continuous vector space where Cosine Similarity measures semantic alignment.

1.5 Multi-Step LLM Reasoning Chain: Chain-of-Thought (CoT) Prompting

- Where it is used: Inside the Qwen3-8B triaging model.
- What it does: Sequentially breaks down the triage task into 4 sub-steps (summarization ->

structural parameter extraction -> duplicate comparison -> workaround extraction). It forces the LLM to output its reasoning path before generating the duplicate decision, reducing hallucinations.

1.6 Workflow Routing: State-Aware Decision Matrix

- Where it is used: The final verdict routing step.
- What it does: A deterministic rule-based matrix that combines the LLM's similarity classification with the live lifecycle status (Open vs. Resolved) of the duplicate ticket in Jira to determine downstream actions (DUPLICATE, WORKAROUND_AVAILABLE, or NEW_ISSUE).

2. Template Fingerprinting Repair Script

2.1 The Code Excerpt (from fix_ground_truth.py)

```
ENVIRONMENTS = [
    "powerstore-lab-01", "powerstore-lab-02", "san-testbed-qa-03",
    "nvme-cluster-dev", "fc-matrix-env", "iscsi-regression-lab",
    "replication-testbed", "metro-cluster-env", "k8s-san-lab",
    "vmware-integration-env",
]

def fingerprint(title: str) -> str:
    t = title.lower()
    for env in ENVIRONMENTS:
        t = t.replace(env, "")
    t = re.sub(r'\b[a-z]*d[a-z0-9]{3,}\b', '', t)      # Strip variable IDs
    t = re.sub(r'\bd+(\.\d+)*%\b', '', t)           # Strip numeric parameters
    t = re.sub(r'\b(on|in|at|for)\s*$', '', t.strip())
    t = re.sub(r'\s+', ' ', t).strip()
    return t
```

2.2 Algorithm Classification

- Classification: A Deterministic, Rule-Based Text Normalization and Regular Expression (Regex) Filtering Algorithm.
- Objective: Converts variable error strings containing dynamic parameters (such as environment names, pool IDs, version numbers, and percentages) into a standardized signature (the "fingerprint") representing the underlying software log template.
- Role in Project: Used in fix_ground_truth.py to repair the mismatched synthetic dataset annotations. By aligning logs and tickets based on fingerprint identity, keyword Jaccard overlap on true duplicate pairs was improved from 0.045 to 0.784, resolving duplicate detection learning blocks.

3. Comprehensive Examiner Q&A Guide

Q3.1: Why did you use Retrieval-Augmented Generation (RAG) instead of fine-tuning a classifier model?

- Answer:
- Data Scarcity: Training a classifier from scratch requires thousands of labelled failures per class. In production pipelines, bug categories change dynamically, and we only have a limited set of historical tickets.
- Dynamic Knowledge updates: If a new bug is filed or resolved in Jira, a RAG system indexes it instantly. A fine-tuned classifier would require expensive retraining cycles to learn about the new ticket.
- Actionable Remediation: Classification models only output a category label (e.g., Class 42). RAG retrieves the actual Jira ticket description and workaround field, allowing the LLM to extract concrete steps for the developer.

Q3.2: How does the overall pipeline flow from a Jenkins failure to a developer verdict?

- Answer: The pipeline is split into offline and online phases:
1. Offline Ingestion: Jira tickets are preprocessed into text blobs, embedded using a hybrid model, and stored in a local Qdrant collection.
 2. Online Triage:
 - The raw Jenkins log is stripped of ANSI escape codes, split into stage chunks, and filtered for error patterns.
 - Prompt 1 summarizes the failure; Prompt 2 extracts structured fields like error messages and components.
 - Hybrid Search queries Qdrant using dense semantic vectors and sparse lexical vectors, returning the top-5 tickets.
 - Prompt 3 compares the failure against the candidates to judge if it is a duplicate.
 - State-Aware Decision Engine inspects the candidate's status to route the ticket to DUPLICATE, WORKAROUND_AVAILABLE, or NEW_ISSUE.
 - Prompt 4 extracts the workaround list if the action is WORKAROUND_AVAILABLE.

Q3.3: Why did you upgrade from dense-only semantic search to a hybrid search?

- Answer:
- Semantic Compression Loss: Transformer models (like BGE-large) compress text into dense vectors. While they excel at paraphrase similarity ("database down" $\approx$ "SQL connection timed out"), they compress rare alphanumeric tokens (e.g., error codes like

ISCSI_ERR_TCP_CONN_CLOSE or ticket IDs like PSTR-1242) into vaguely similar vectors.

- Keyword Precision: Sparse retrieval (BM25) preserves exact-match keyword signals.
- The Hybrid Solution: By combining both, we get the best of both worlds: semantic understanding of symptoms (dense) + exact-match accuracy for trace identifiers and error codes (sparse).

Q3.4: What is Reciprocal Rank Fusion (RRF), and why is it used instead of simply adding dense and

- Answer:
- Score Incompatibility: Cosine similarity (dense) is bounded in $[-1, 1]$ (or $[0, 1]$ normalized), whereas BM25 (sparse) scores are unbounded and depend on document length and term frequency. Adding them directly is mathematically invalid.
- Rank-Based Fusion: RRF solves this by ignoring raw scores and focusing only on the rank position of a document in each retrieval branch. The RRF formula is:

$$\text{RRF_Score}(d) = \sum_{m \in M} \frac{1}{k + r_m(d)}$$

where M is the set of retrieval strategies (dense and sparse), $r_m(d)$ is the rank of document d in strategy m , and k is a constant (typically 60) that smooths the impact of low-ranked items.

- This ensures a document ranked #1 in both strategies is prioritized without needing complex score calibration.

Q3.5: What are the specific roles of the 4 prompts in the RAG chain?

- Answer:
- Prompt 1 (Summarize): Condenses raw, noisy console output into a 3-sentence summary highlighting the failing component, error type, and symptoms.
- Prompt 2 (JSON Extraction): Extracts structured fields (bug_title, component, error_message, symptoms, category) to standardize input for the retrieval database.
- Prompt 3 (Duplicate Verdict): Acts as the decision judge, comparing the structured new bug fields against the top-K retrieved Jira tickets to determine if there is a match.
- Prompt 4 (Workaround Extraction): Parses resolved tickets to extract a clean, numbered list of actionable developer steps.

Q3.6: How does the State-Aware Decision Engine operate, and why can't the LLM make the final acti

- Answer:
- Stateful Rules: The LLM's role is to verify content similarity (is this failure the same bug as that ticket?). The final business action depends on the state of the ticket in Jira, which the LLM shouldn't have to guess or hardcode.

- The Matrix: The engine routes the action programmatically:
- If a duplicate is found and its status is Open/In Progress, it routes to DUPLICATE (developers should link/subscribe to the existing ticket).
- If a duplicate is found and its status is Resolved/Closed, it routes to WORKAROUND_AVAILABLE (developers get immediate instructions on how to bypass the failure).
- If no duplicate exists above the similarity threshold, it routes to NEW_ISSUE (drafts a new bug report).
- This decouples logical comparison (LLM) from state-based workflow rules (deterministic code).

Q3.7: How did the threshold sweep differ between "demo" mode and "production" mode?

- Answer:
- Production Mode: Sweeps cosine similarity from BAAI/bge-large-en-v1.5 across $[0.70, 0.95]$. This is a standard similarity metric representing document proximity in vector space.
- Demo Mode: Uses RRF rank scores. Because RRF score ranges depend on rank $(\frac{1}{60+r})$ and are bounded by the number of branches (max ≈ 0.0328), sweeping $0.70-0.95$ would result in 0% duplicate predictions. Demo mode instead sweeps fractions of the maximum possible RRF score (15% to 75% , or ≈ 0.0049 to 0.0246) to ensure evaluation logic works end-to-end.

Q3.8: How did the different prompting strategies (zero-shot, few-shot, CoT) perform, and what did it

- Answer:
- Zero-shot: Directly asks if the failure is a duplicate. It runs fastest but struggles with nuanced differences (e.g., same component, different error codes).
- Few-shot: Includes 2-3 examples of duplicate and non-duplicate decisions in the system prompt. It improves precision but increases context size.
- Chain-of-Thought (CoT): Instructs the model to generate its step-by-step reasoning before outputting the final JSON verdict (e.g., comparing error codes, components, then symptoms). It produces the highest F1 score by ensuring the LLM evaluates structural matches rather than surface-level terminology.

Q3.9: Why did you choose Qwen3-8B and BAAI/bge-large-en-v1.5?

- Answer:
- Data Privacy & Compliance: CI/CD logs and internal Jira tickets contain sensitive infrastructure names, IP addresses, and code snippets. Using local models via Ollama and sentence-transformers guarantees that no data is sent to external APIs (like OpenAI),

adhering to corporate security guidelines.

- Qwen3-8B: Top-tier open-source reasoning model under Apache 2.0. It excels at JSON extraction, reasoning, and follows system instructions reliably on consumer hardware.
- BGE-Large-en-v1.5: Ranked highly on the MTEB (Massive Text Embedding Benchmark) for retrieval tasks, providing 1024-dimensional semantic coverage.

Q3.10: Why did you choose a local, file-based Qdrant client over a running Docker container or Qdrant Cloud?

- Answer:
- Simplicity & Resource Constraints: For a dissertation prototype, running a separate Docker service increases setup friction.
- Local Storage: Qdrant's file-based storage path (`qdrant_storage/`) runs natively inside Python, allowing database persistence and indexing without requiring external port bindings or service management. In production, this can be seamlessly swapped to a server-client model by changing the client initialization URL.